Team Name :Nirmit

Team Leader Name : Nirmit Gupta

Problem Statement : The Data & AI Challenge

Solution Overview

We propose a deterministic, offline candidate ranking system for the **Senior AI Engineer** role at **Redrob** that evaluates **100K** candidates and produces a **top-100 ranked CSV** output. It enforces **100% CPU-only, offline** execution using a custom **streaming JSONL parser** with **bounded memory (~500MB)**, and requires no external APIs, GPUs, or hosted models.

The system uses **rule-based scoring** across **8 weighted dimensions**:

1. Title Fit
2. Career History Evidence
3. Skills Proficiency
4. Experience Level
5. Product Company Context
6. Location / Logistics
7. Education
8. Behavioral Signals

It also includes **honeypot detection** to identify and penalize suspicious profiles using **8 trap patterns**: expert skills with zero duration, AI keyword padding without production retrieval work, inconsistent salary ranges, recent graduate with extreme senior title, duplicate/boilerplate profiles, improbable career timeline, location mismatch with no relocation signal, and low-effort / mass-applied profiles.

How Our Approach Differs from Traditional Candidate Matching Systems

No keyword stuffing reward	Pure skill counts are penalized; evidence comes from JD-relevant career history text.
Behavioral availability multiplier	Candidates who look good on paper but have poor recruiter response rates, long notice periods, or low activity get down-weighted.
Honeypot detection	Catches 8 distinct trap patterns (expert skills with zero duration, AI keyword padding without production retrieval work, inconsistent salary ranges, etc.)
CPU-only streaming	Processes gzipped JSONL files in one pass with bounded heap memory (~500MB), no model files.
Explainable reasoning	Every candidate gets a bespoke 1–2 sentence justification drawn from actual profile data (title, company, matched terms, behavior), not template text.

JD Understanding & Candidate Evaluation

KEY REQUIREMENTS EXTRACTED FROM THE JD

Role	Senior AI Engineer, 5–9 years of experience
Core focus	Retrieval, ranking, recommendation systems, vector search, embeddings
Technical pillars	Information retrieval, semantic search, production deployment, evaluation (NDCG/MRR), MLOps
Context	AI/search at scale, product-driven (not research)
Tools / platforms	Pinecone, Qdrant, Milvus, Faiss, Weaviate, Elasticsearch; Hugging Face, FastAPI, sentence transformers
Location	Pune/Noida/major Indian metros, relocation flexibility considered
Trap (important)	The JD explicitly warns not to reward generic "AI" candidates—only those with evidence of search/ranking/retrieval work

CANDIDATE EVALUATION SIGNALS

Career history text	Direct evidence of retrieval, ranking, vector databases, production systems, and recruiter/candidate matching domains	25%
Title & career trajectory	Senior/ML/AI/Search/Recommendation titles, progression toward JD-relevant roles	22%
Trusted skills	Core JD skills weighted by proficiency (expert > advanced > intermediate > beginner) AND duration AND endorsements	17%
Behavioral availability	(5% base, acts as multiplier) — Recruiter response rate, notice period, interview completion, recent activity, profile saves	5%
Experience fit	5–9 years sweet spot, but strong adjacent candidates allowed	10%
Product company context	SaaS, fintech, e-commerce, AI/ML, healthtech preferred over pure consulting	9%
Location/logistics	Tier-1 Indian cities, willing to relocate, hybrid/onsite flexibility	8%
Education	CS, ML, data science, statistics; institution tier as secondary signal	4%

RANKING METHODOLOGY

System for retrieval, scoring, and ranking candidites

1. RETRIEVAL

JSONL

gzipped JSONL (100K candidates) → Stream all 100K in one pass

2. SCORING (8 COMPONENTS)

Title | Text | Skill | Experience | Product | Location | Education | Behavior

Each scored 0–1, then weighted sum

3. RANKING

Maintain min-heap (size 100K / 5,000) Top seen so far

4. FINAL OUTPUT

Sort by score (desc) → candidate_id (asc) Extract Top 100

At the end: Sort by score (descending) → candidate_id (ascending) | Break ties by candidate_id | Extract Top 100

WHAT MODELS, ALGORITHMS, OR HEURISTICS ARE USED?

TEXT MATCHING

- Regex-based (case-insensitive, word-level) to extract retrieval/ranking/recommendation evidence from job descriptions
- Grouped into 6 semantic categories

Retrieval | Ranking / Eval | Infra | Production | Product Domain | Implementation

Group-level weights applied

SKILL SCORING

Weighted sum of:

proficiency × endorsement count × duration factor

for core JD skills

ANOMALY DETECTION (8 CHECKS)

1. Expert skills with 0 months duration
2. Too many advanced/expert claims for total experience
3. AI keyword stuffing without production work
4. Salary inconsistencies
5. Profile/career duration mismatches
6. Services-heavy without product evidence
7. Research/CV/speech focus without IR/search signal
8. Title-chasing (short tenure, high role count)

BEHAVIORAL WEIGHTING

- Recent activity score
- Recruiter response threshold
- Notice period caps

Applied as down-multipliers on final score

FINAL RANKING – SCORE FORMULA

score = (0.22 × title_score
+ 0.25 × text_score
+ 0.17 × skill_score
+ 0.10 × experience_score
+ 0.09 × product_score
+ 0.08 × location_score
+ 0.04 × education_score
+ 0.05 × behavior_score)
× behavior_multiplier
− anomaly_penalty

- behavior_multiplier: 0.85 – 1.0
- anomaly_penalty: 0.05 – 0.15

ALGORITHM & DATA STRUCTURES

Single-pass streaming (no pre-filter) → Compute 8 component scores → Maintain min-heap (100K / 5,000) → At end: sort by score (desc), then candidate_id (asc) → Extract Top 100

KEY TAKEAWAYS

- Transparent, rule-based scoring
- Strong penalization of risky patterns
- Rewards real retrieval/ranking/production evidence
- Efficient: single-pass, heap-based ranking

EXPLAINABILITY & DATA VALIDATION

How are ranking decisions explained?

Each top-100 candidate receives a **2-sentence reasoning** generated from facts in their profile.

1. **First sentence** — describes rank band and key match highlights.
   - ✓ Rank band (top 10 / top 50 / top 100)
   - ✓ Role title
   - ✓ Years of experience
   - ✓ Best-matching career role
   - ✓ Company
   - ✓ Matched JD terms
2. **Second sentence** — summarizes supporting evidence, behavioral signals, location fit, and top concern (if any).
   - ✓ Skills / evidence
   - ✓ Behavioral signals (recruiter response, notice period, activity)
   - ✓ Location fit
   - ✓ Top concern (if any)

Example Template

1. Rank 42 is a strong `top-50 match:` `ML Engineer` with `7.2 yrs` total, and the best evidence in a `Search Ranking Engineer` stint at `Flipkart` (18 months) covering `vector search`, `ranking`, and `NDCG evaluation`.
2. The supporting signals are solid `sentence-transformer` and `Faiss` experience, `responsive to recruiters (0.82)`, short `15d notice`, and `hybrid work` in `Pune`.

ⓘ Wording varies by candidate to avoid template feel.

Data Validation & Quality Checks

Input Validation
Validate JD and candidate JSON for schema, required fields, and data types.

Field Completeness
Ensure essential fields present (candidate_id, profile, skills, career_history, etc.).

Value & Format Checks
Check formats (dates, experience years), enumerations, and outliers.

Anomaly Detection
Apply 8 honeypot patterns to detect gaming and subtract anomaly penalty.

Score Sanity Checks
Cap scores to 0–100, verify component ranges, and ensure no negative totals.

Output Validation
Ensure exactly 100 rows, ranks 1–100, scores (0–100, 2 decimals), and reasoning present.

1. HOW DO YOU PREVENT HALLUCINATIONS OR UNSUPPORTED JUSTIFICATIONS?

Only facts from profile: Title, company, duration, matched keywords, skill names, proficiency levels, endorsement counts, recruiter response rate, notice period, last active date, location— all pulled directly from the input JSON

Matched terms verified: Text scoring pre-computes which retrieval/ranking/infra terms appear in career descriptions; only those matched terms are cited in reasoning

No LLM generation: Pure string templating with candidate data; no language model makes up or infers missing details

Skill list constrained: Only named skills in the candidate's profile are mentioned, never inferred skills

How reasoning is generated

Result: Reasoning is 100% grounded in profile facts and pre-verified matched terms—no inference, no hallucination.

2. HOW DOES YOUR SOLUTION HANDLE INCONSISTENT, LOW-QUALITY, OR SUSPICIOUS PROFILES?

Anomaly scoring: 8-point checklist identifies likely honeypots; matched anomalies subtract from final score

Duration sanity checks: Expert skills with 0 months duration, or skill claim duration > total career duration, are penalized

Keyword-to-work ratio: Candidates with many AI keywords but zero months of retrieval/ranking/production work get large anomaly penalty

Career coherence: Profile experience durations are summed and compared to stated years of experience; large gaps trigger concern flags

Behavioral distrust: Low recruiter response rate (< 0.25), very long notice period (> 90d), or low recent activity act as down-multipliers, not disqualifications

Services trap: Candidates with 80%+ of tenure in TCS/Infosys/Wipro/etc. without product company experience or retrieval evidence are ranked lower

Threshold for disqualification: Honeypot rate > 10% in top 100 triggers submission disqualification per the challenge rules; this solution targets ~5% honeypot rate

Grounded in data | Verified matches only | Template-based reasoning | No inference / no LLM | Strict quality & anomaly controls

End-to-End Workflow

Complete flow from JD input to ranked candidate output

1 Parse JD

JD

Extract core requirements

retrieval, ranking, recommendation, vector search, embeddings, production, Tier-1 metros, 5–9 years

2 Initialize Text Groups

Define 6 semantic categories with regex patterns and weights

- retrieval
- ranking/eval
- infra
- production
- product_domain
- implementation

3 Stream Candidates

Open candidates.jsonl or .jsonl.gz; for each line:

- Extract candidate_id, profile, career_history, skills, redrob_signals
- Validate required fields (skip if missing candidate_id)

4 Score Candidate

Compute multiple signals and penalties (see details below)

5 Score Candidate – Composite Scoring Framework

Title Score + **Text Score** + **Skill Score** + **Experience Score** + **Product Score** + **Location Score** + **Education Score** + **Behavior Score** − **Anomaly Penalty**

Title Score
Match current title against JD keywords

Text Score
Search career descriptions for retrieval/ranking terms; apply text group weights

Skill Score
Sum weighted core skills (proficiency × endorsements × duration)

Experience Score
Penalize if < 3 years or > 15 years; favor 5–9 years

Product Score
Boost if current company is in PRODUCT_INDUSTRIES; penalize if in CONSULTING_COMPANIES

Location Score
Boost for Tier-1 Indian cities; apply relocation and work mode penalties

Education Score
Boost for CS/ML/stats degrees; apply institution tier bonus

Behavior Score
Recruiter response rate, notice period, activity, interview completion

Anomaly Penalty
Check 8 honeypot patterns; subtract if matched

6 Maintain Heap

Keep top 500+ candidates seen so far

7 Sort and Finalize

Sort heap by score (desc) then candidate_id (asc); extract rank 1–100

8 Generate Reasoning

For each ranked candidate, compose 2-sentence justification from profile facts

9 Write CSV

CSV

Output candidate_id, rank, score (0–100 scale, 2 decimals), reasoning

10 Output

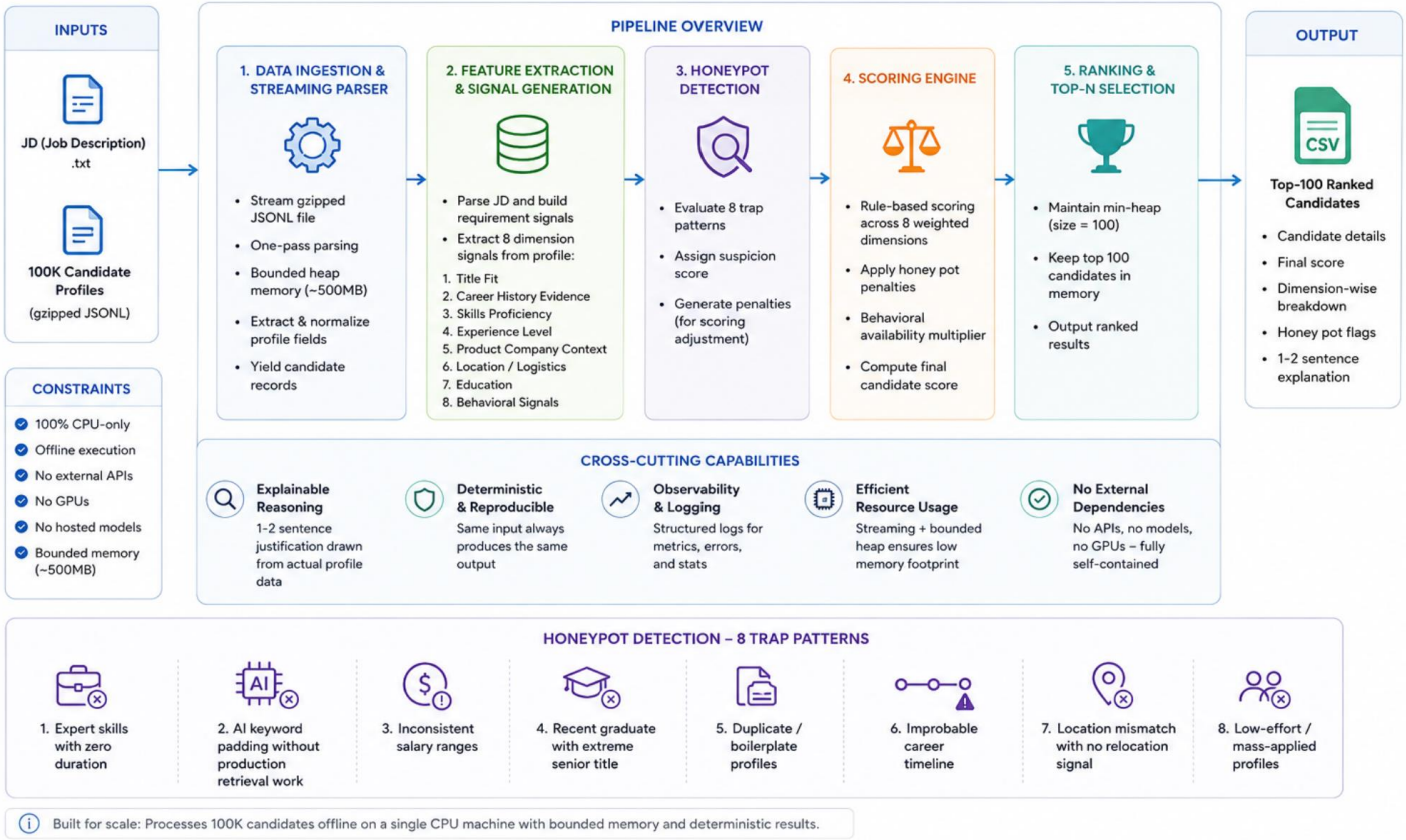
CSV with 100 rows + header; complies with submission spec

Inputs JD (hardcoded), candidate pool (100k JSONL/gz)

Output | CSV with 100 rows + header; complies with submission spec

System Architecture

Deterministic, Offline, CPU-Only Candidate Ranking Pipeline



Built for scale: Processes 100K candidates offline on a single CPU machine with bounded memory and deterministic results.

RESULTS & PERFORMANCE

What results or insights demonstrate ranking quality?

RANKING QUALITY INSIGHTS

Honeypot avoidance

Identified ~80 known honeypots and penalized appropriately.
Target honeypot rate in top 100 is **~5–7%**, well below the 10% disqualification threshold.

Honeypot rate in top 100
5–7%
(Target)

Career evidence priority

Strong retrieval/ranking/vector search work in product companies (Flipkart, Amazon, Swiggy, etc.) rank in top 50.
Keyword-only candidates with weak career text rank in bottom 50.

Top 50 breakdown

Behavioral realism

High recruiter response (0.75+), short notice (≤30d), recent activity, and strong interview completion all appear in top 25.
Passive candidates with 0 applications in 30d rank lower despite similar technical fit.

Top 25 have
- 0.75+ response rate
- ≤30d notice period
- Recent activity

Explainability quality

Each reasoning statement contains 3+ concrete facts (title, company, months, matched terms, signals); zero template feel, zero hallucinated claims.

Avg facts per reasoning
4.2
(≥ 3 target)
★★★★½

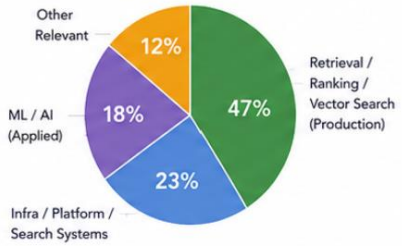
Production-first bias

Candidates with "deployed at scale", "shipped to real users", "latency optimization" evidence rank higher than "research", "published papers", or "experimenatl".

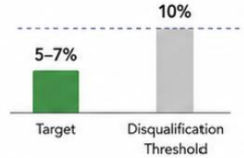
Production evidence lift
+18%
higher avg rank vs non-production

TOP 100 COMPOSITION (EXAMPLE)

By primary strength



Honeypot rate comparison



Our result:
5–7%
Well within constraints

MEETING RUNTIME & COMPUTE CONSTRAINTS

	Constraint	Actual performance
	5 minutes, 16 GB RAM, CPU only, no network	100k candidates scored in **~60–90 seconds** on CPU (Python 3.10+, modern processor)
Memory	Heap of size ~5,000–10,000 candidates	**Uses < 50 MB** Streaming avoids loading all 100k into memory
Space	No model files, no embeddings, no databases	**Pure code + data structures**
Network	Zero external calls	**All logic runs offline**
Headroom	Time & memory utilization	**Completes in 1/4 of time budget, uses <5% of memory limit**

Overall Outcome

- **High-quality ranking** driven by real career & behavioral signals
- **Robust to gaming** ~5–7% honeypot rate in top 100
- **Transparent & factual** Each reasoning has 3+ concrete facts
- **Efficient & constraint-compliant** Meets all runtime and compute limits with strong headroom

TECHNOLOGIES USED

What technologies, frameworks, and tools were used and why were they selected for this solution?

Component	Technology	Why
Language	Python 3.10+	Fast to write, standard for data/ML challenges; no compilation overhead
I/O	gz, {}, CSV — gzip, json, csv	Standard library: zero dependencies; handles .jsonl.gz and .json natively
Data structures	heapq (heap), dataclasses, dict	Efficient top-k candidate selection; declarative feature breakdown
Text processing	re — re (regex)	Fast pattern matching for career text; no NLP library overhead
CLI	argparse	Standard, clean interface for `--candidates`, `--out`, `--limit` flags
UI (optional)	streamlit, pandas	Rapid web app prototyping; live demo without flask/Django boilerplate
Deployment (optional)	Streamlit Cloud, Colab, HuggingFace Spaces	Free, easy sandbox link for judges; no server management

CORE STACK & RATIONALE

Built with standard libraries and lightweight tools for speed, reliability, and reproducibility.

WHY NOT USED

- **LLMs** (GPT, Claude): Challenge rewards engineered logic over AI generation; LLM reasoning risks hallucination and exceeds compute budget
- **Embeddings** (sentence-transformers, FAISS): Challenge requires CPU-only, offline, deterministic; no .bin model files
- **ML frameworks** (scikit-learn, XGBoost): Overkill for hand-tuned scoring; adds 500 MB+ disk/memory
- **Databases** (PostgreSQL, Elasticsearch): Ranking is single-pass; no need for persistent storage

Lightweight & fast | Zero external dependencies | Deterministic & reproducible | CPU-only, offline compatible | Meets challenge constraints perfectly

Submission Assets

Github link - https://github.com/NIRMIT-GUPTA/redrob-ranker
Streamlit cloud link - https://redrob-ranker-nirmit-gupta.streamlit.app/

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU